

# EC200A-CN(TA) QuecOpen SELinux 应用指导

车规级模块系列

版本：1.0.0

日期：2022-08-24

状态：临时文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司  
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233  
电话：+86 21 5108 6236 邮箱：[info@quectel.com](mailto:info@quectel.com)

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登陆网址：  
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：[support@quectel.com](mailto:support@quectel.com)。

## 前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

## 使用和披露限制

### 许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

### 版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

### 商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

### 第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他软硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

## 免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2022，保留一切权利。

**Copyright © Quectel Wireless Solutions Co., Ltd. 2022.**

## 文档历史

### 修订记录

| 版本    | 日期         | 作者       | 变更描述 |
|-------|------------|----------|------|
| -     | 2022-08-24 | Ditto YU | 文档创建 |
| 1.0.0 | 2022-08-24 | Ditto YU | 临时版本 |

## 目录

|                             |           |
|-----------------------------|-----------|
| 文档历史 .....                  | 3         |
| 目录 .....                    | 4         |
| 表格索引 .....                  | 5         |
| <b>1 引言 .....</b>           | <b>6</b>  |
| <b>2 SELinux 简介.....</b>    | <b>7</b>  |
| 2.1. SELinux 概述.....        | 7         |
| 2.2. SELinux 运行条件 .....     | 7         |
| <b>3 系统配置.....</b>          | <b>8</b>  |
| <b>4 文件标签.....</b>          | <b>10</b> |
| 4.1. 文件标签介绍.....            | 10        |
| 4.2. 文件标签制作.....            | 10        |
| <b>5 SELinux 策略.....</b>    | <b>12</b> |
| 5.1. 策略构成 .....             | 13        |
| 5.1.1. 客体类别和权限集 .....       | 13        |
| 5.1.2. 角色与用户 .....          | 14        |
| 5.1.3. 类型 .....             | 14        |
| 5.2. 策略编写语法介绍 .....         | 15        |
| 5.2.1. 常见访问向量规则关键字 .....    | 15        |
| 5.2.2. 安全上下文转换 .....        | 15        |
| <b>6 策略编写及使用实例.....</b>     | <b>17</b> |
| 6.1. 设置 SELinux 配置文件 .....  | 17        |
| 6.2. 文件标签编写.....            | 17        |
| 6.3. 策略规则文件编写 .....         | 18        |
| 6.3.1. 类型文件编写.....          | 18        |
| 6.3.2. TE 规则文件编写 .....      | 18        |
| 6.4. 编译及使用 .....            | 18        |
| 6.4.1. 编译文件系统镜像 .....       | 18        |
| 6.4.2. 烧录进模块 .....          | 18        |
| 6.5. 检查策略是否生效 .....         | 19        |
| <b>7 附录 参考文档及术语缩写 .....</b> | <b>20</b> |

## 表格索引

|                 |    |
|-----------------|----|
| 表 1: 参考文档 ..... | 20 |
| 表 2: 术语缩写 ..... | 20 |

Preliminary Document  
Not Checked

# 1 引言

移远通信 EC200A-CN(TA)模块支持 QuecOpen®方案；QuecOpen®是开源的基于 Linux 的嵌入式开发平台，可简化 IoT 应用的软件设计和开发过程。有关 QuecOpen®的详细信息，请参考文档 [1]。

本文档主要介绍在 QuecOpen®方案下，如何运行 SELinux，包含系统配置、文件标签制作和策略编写等相关信息，同时结合实例说明如何运行和验证 SELinux。

## 2 SELinux 简介

### 2.1. SELinux 概述

SELinux (Security-Enhanced Linux) 指安全增强型 Linux，是一个 Linux 内核的安全模块，其提供了访问控制安全策略机制，包括强制访问控制；SELinux 为每个进程与文件都打上一个安全上下文标签，通过该标签实现主体（进程）对客体（资源文件）的访问控制。

### 2.2. SELinux 运行条件

在使用 SELinux 之前，需满足以下几项条件：

- 系统配置：在 SELinux 配置文件 `/etc/selinux/config` 中必须初始化 SELinux 工作状态；详见第3章。
- 文件标签：每个文件都需要有特定的安全上下文标签，SELinux 无法管理没有标签的文件；详见第4章。
- SELinux 策略：在系统 `/etc/selinux/quectel/policy` 目录下必须有 SELinux 的策略文件；详见第5章。



## 3 系统配置

SELinux 配置文件在系统中的路径为`/etc/selinux/config`，在 SDK 包中的路径为`/selinux/config`。配置文件的主要内容如下：

```
# This file controls the state of SELinux on the system.
# SELINUX= can take one of these three values:
# enforcing - SELinux security policy is enforced.
# permissive - SELinux prints warnings instead of enforcing.
# disabled - No SELinux policy is loaded.
SELINUX=permissive
# SELINUXTYPE= can take one of these two values:
# default - equivalent to the old strict and targeted policies
# mls - Multi-Level Security (for military and educational use)
# src - Custom policy built from source
SELINUXTYPE=quectel
# SETLOCALDEFS= C
```

该配置文件用于配置 SELinux 的环境变量，具体变量含义如下：

1. **SELINUX** 变量用于初始化 SELinux 工作状态，支持配置以下三种工作模式：

- **enforcing**：强制模式；在此模式下，如果当前执行的操作不满足 SELinux 策略，不满足操作权限的日志会被记录，并且一切不满足安全策略的系统行为会被拒绝。
- **permissive**：宽容模式；在此模式下，如果当前执行的操作不满足 SELinux 策略，不满足操作权限的日志会被记录，并且不满足安全策略的系统行为不会被拒绝。
- **disabled**：关闭 SELinux 功能

2. **SELINUXTYPE** 变量决定了 SELinux 加载策略时，在哪个目录寻找策略文件。支持的取值如下：

- **mls** 表示多级别访问控制；预留值，暂不支持配置。
- **src** 表示系统初始化时，SELinux 加载策略文件的位置。例如，若将其设置为 **quectel** 则表示 SELinux 将在`/etc/selinux/quectel/policy`目录下寻找策略文件。
- **default** 暂不支持。

建议将 SELinux 策略置于某个目录之下，方便查询和控制，此处只关注 **src** 即可，在上面提供的配置文件中，**src** 被显式设置为“**quectel**”。

3. **SETLOCALDEFS** 为预留变量，暂不支持。

## 备注

*permissive*（宽容模式）仅供调试使用。移远通信提供的`/etc/selinux/config`文件，默认将 *SELINUX* 设置成 *permissive*，客户可按需修改为其他模式。

Preliminary Document  
Not Checked

# 4 文件标签

## 4.1. 文件标签介绍

SELinux 文件标签用于设置客体资源的安全上下文。为了使镜像文件支持 SELinux 功能，在镜像文件的制作过程中，需要使用移远通信修改制作的 **setfiles** 工具给镜像文件打上 SELinux 标签，使用 **mksquashfs4** 工具制作出包含 SELinux 标签的文件系统镜像。镜像制作可参考第 6.4.1 章，部分脚本命令如下所示：

```
setfiles -r /ql-ol-rootfs /ql-ol-rootfs/etc/selinux/quectel/contexts/files/file_contexts /ql-ol-rootfs
mksquashfs4 /ql-ol-rootfs /target/root.squashfs -noappend -root-owned -comp xz -Xpreset 8 -Xe -Xlc 0
-Xlp 2 -Xpb 2 -Xbcj arm -b 64k -p '/dev d 755 0 0' -p '/dev/console c 600 0 0 5 1' -processors 1
```

*file\_contexts* 为文件标签配置文件，客户可以手工制作。制作方法，请参考第 4.2 章。

## 4.2. 文件标签制作

*file\_contexts* 为所有 SELinux 文件安全标签的配置文件；所述 SELinux 文件包括镜像文件与系统动态创建的文件。在 **mkfs.ubifs** 命令行中必须添加 **file\_contexts** 的路径，当版本包烧录进模块后，此文件将位于 */etc/selinux/quectel/contexts/files* 目录下。*file\_contexts* 语法如下：

```
regex <-type> ( <file_label> | <<none>> )
```

regex: 正则表达式

<-type>: 类型，取值如下：

- : 只匹配普通文件
- d: 只匹配目录
- l: 只匹配符号链接文件
- c: 只匹配字符设备文件
- b: 只匹配块设备文件

不指定类型：匹配任何文件

<file\_label>: 安全上下文字符串

<<none>>: 忽略不设置

举例如下：

```
/* u:object_r:rootfs:s0
```

```
/bin/ -d      u:object_r:bin:s0
/bin/.*      u:object_r:bin_exec:s0
/bin/sh -l    u:object_r:shell_exec:s0
```

*file\_contexts* 将逐一匹配上述举例中的内容；根据正则表达式：

- */text.txt* 文件匹配 */\** 成功，后续匹配都失败，因此 */text.txt* 安全上下文会被设置为 **u:object\_r:rootfs:s0**；
- */bin/dmesg* 文件匹配 */\** 成功，匹配 */bin/\** 也成功，根据逐一匹配原则，会以最后匹配成功的一行来设置上下文，因此 */bin/dmesg* 安全上下文会被设置为 **u:object\_r:bin\_exec:s0**。

## 备注

当编写 *file\_contexts* 文件时，待设置的安全上下文需与第 5.1.3 章所述类型一致。*file\_contexts* 设置的安全上下文由策略定义，否则无意义。

# 5 SELinux 策略

SELinux 策略是 SELinux 在运行时，判断是否允许主体进程对客体资源执行操作时所依据的准则。SELinux 策略包括客体类别和权限集、用户、角色、类型、多级权限、条件控制等。

SELinux 策略文件框架如下所示：

|  |                  |                                                       |
|--|------------------|-------------------------------------------------------|
|  | app              |                                                       |
|  |                  | quectel.....该目录下的文件定义移远进程的权限策略                        |
|  |                  | system.....该目录下的文件定义系统进程的权限策略                         |
|  | class            |                                                       |
|  |                  | classes.....定义 SELinux 客体类型                           |
|  |                  | perms.....定义 SELinux 客体类型的权限集                         |
|  | macros           |                                                       |
|  |                  | global_macros.....全局宏，定义客体类型权限组                       |
|  |                  | mls_macros.....多级权限宏定义                                |
|  |                  | neverallow_macros.....定义 neverallow 权限宏               |
|  |                  | te_macros.....策略文件编写需要的宏                              |
|  | Makefile.....    | 编译策略文件                                                |
|  | misc             |                                                       |
|  |                  | fs_use.....定义文件系统的扩展属性，如果不定义，文件系统则不能设置安全上下文           |
|  |                  | genfs_contexts....设置文件系统中文件的默认安全上下文，设置 proc 文件系统安全上下文 |
|  | mls              |                                                       |
|  |                  | mls.....定义 SELinux 多级权限策略                             |
|  | roles            |                                                       |
|  |                  | roles.....定义 SELinux 角色                               |
|  | sid              |                                                       |
|  |                  | initial_sid_contexts.....初始化缺省安全上下文                   |
|  |                  | initial_sids.....定义缺省安全上下文序列                          |
|  | type             |                                                       |
|  |                  | attributes.....定义 SELinux 的类型属性                       |
|  |                  | device.te.....定义 device 相关的 SELinux 类型                |
|  |                  | file_type.....定义文件的 SELinux 类型                        |
|  | users            |                                                       |
|  |                  | users.....定义 SELinux 用户                               |
|  | users_extra..... | 定义策略前缀条目                                              |

SELinux 控制的主体是进程，它限制进程对资源的访问权限；客体就是资源，包括文件、设备、网络、信号和进程等。每个主体与客体都必须有一个安全上下文，该安全上下文决定了主体进程对客体资源是否

有访问权限。

## 5.1. 策略构成

### 5.1.1. 客体类别和权限集

客体就是主体要访问的内容，在 SDK 路径 `/ql-selinux/class/classes` 中定义。客体权限集中，客体不同，权限也不同，比如，如果客体是文件，那么文件有创建、打开、读、写和执行等操作；如果客体是 `socket`，那么客体有绑定、监听、接受和连接等操作。移远通信已经完成关键客体权限的定义，可根据需求增加。客体类别定义语法如下：

```
#类别定义语法
class class_name
class: 类别定义关键字
class_name: 要定义的类别标识符（可以是字母、数字和下划线）
示例：
class file
class dir
class chr_file
#类别权限集语法
common common_name {perm_set}
common: 权限集定义关键字
common_name: 权限集标识符
perm_set: 权限集
示例：
common file_perm
{
read
write
ioctl
}
#客体类别与权限集相关联
class class_name [inherits common_name] [{perm_set}]
class: 类别定义关键字（并非定义一个新类别，而是之前已定义的类别）
class_name: 要定义的类别标识符（并非定义新的类别标识符，而是之前已定义的标识符）
inherits: 关键字，代表一个类别继承一个权限集
common_name: 权限集标识符（并非定义新的权限集标识符，而是之前已定义的标识符）
perm_set: 继承的权限集如果不够，还可以添加新的权限
示例：
class chr_file
inherits file_perm
{
execute_no_trans
```

```
entrypoint
execmod
open
audit_access
}
```

### 5.1.2. 角色与用户

以 `file_contexts` 的安全上下文 `u:object_r:bin_exec:s0` 为例，其中 `u` 表示用户，`object_r` 表示角色，`bin_exec` 表示类型，`s0` 表示多级安全。模块的 SELinux 中，移远通信已经完成对用户和角色的定义，无需修改；语法如下：

```
#SELinux 角色定义
1.  role role_name;
或
2.  role role_name [types type_set];
role: 角色定义关键字
role_name: 角色标识符
types: 角色关联类型的关键字
type_set: 关联的一个类型
#第一种是直接定义一个角色，第二种是定义角色的同时，关联一个类型
示例:
role r;
role r types domain; #如果前面定义了 r 角色，这里只要关联一个 domain 类型就可以了
#SELinux 用户定义
user user_name roles {role_set};
user: 用户定义关键字
user_name: 用户标识符
roles: 与角色关联的关键字
role_set: 关联的角色集
示例:
user u roles {r};
```

### 5.1.3. 类型

在 SELinux 中最主要的访问控制是 TE（即类型强制访问控制），需要特别关注类型。类型是安全上下文中的第三个成员，即 `u:object_r:bin_exec:s0` 中的 `bin_exec`。比如，在策略中写入 `allow bin_exec data_t:file open read`，则表示 `bin_exec` 类型的进程对 `data_t` 类型的文件有打开和读取操作的权限。类型语法如下：

```
#SELinux 类型定义语法
type type_name [ alias alias_set ] [, attribute_set] ;
type: 类型定义关键字
```

```

type_name: 类型标识符
alias: 定义类型别名的关键字
alias_set: 别名集合
attribute_set: 类型属性集合（属性与类型标识符处在同一命名空间，也就是属性与类型不能重名）
示例：
type bin_exec,file_type,exec_type;#定义一个 bin_exec 类型，有 file_type 与 exec_type 两个属性
#SELinux 类型属性定义
attribute attribute_name;
attribute: 属性定义的关键字
attribute_name: 属性标识符
#注意：属性本质也是类型，可以说它是类型的类型，它类似于一个数组，这个数组中有多个类型。
#类型与属性关联语法
typeattribute type_name attribute_name;
#注意：当我们定义一个新的类型时没有关联某个属性，那么就可以通过 typeattribute 来关联
示例：
type user_file_t;
attribute file_type;
type system_file_t,file_type;#定义类型时直接关联属性
typeattribute user_file_t file_type;#定义类型完成后关联属性
#属性的作用：
allow bin_exec file_type:file {open read};
#这个策略语句可以说明属性的作用，这个语句等价于下面两个语句的组合
allow bin_exec user_file_t:file {open read};
allow bin_exec system_file_t:file {open read};

```

## 5.2. 策略编写语法介绍

### 5.2.1. 常见访问向量规则关键字

策略编写语法中涉及到的常见访问向量规则关键字如下：

- **allow**: 表示允许主体对客体执行的操作；
- **dontaudit**: 表示不记录违反规则的决策信息，且违反规则不影响运行（允许操作且不记录）；
- **auditallow**: 表示允许操作并记录访问决策信息（允许操作且记录）；
- **neverallow**: 表示不允许主体对客体执行指定的操作。

### 5.2.2. 安全上下文转换

对于 TE 访问控制来说，Linux 系统中存在很多个进程，为对不同的进程实现不同的策略控制，这些进程需要具有不同的安全上下文。系统最先启动的用户进程是 `init` 进程，所以对后续启动的进程需要进行安全上下文转换。如果不能进行上下文转换，则 SELinux 没有任何作用。SELinux 是主体进程对客体资源进行访问控制的安全机制。主体进程是随机创建的，例如 `init` 进程可以创建一个 `shell` 进程，如果 `shell` 进程与 `init`



进程保持相同的安全上下文，*shell* 进程和 *init* 进程将具有相同的权限，SELinux 的安全功能将失去作用。所以 *shell* 进程的安全上下文必须与 *init* 进程不同，此时就涉及到安全上下文转换。

安全上下文转换在 *execve* 函数簇调用时完成，即 *execve* 加载可执行文件时，会通过 SELinux 策略检查是否要进行安全上下文转换，如果需要则转换。例如，*init* 进程的类型为 *init\_t*，*shell* 的可执行文件为 */bin/sh*，类型为 *shell\_exec*，现在希望 *init* 进程启动一个 *shell* 时，*shell* 进程类型为 *shell\_t*，那么至少需要满足以下条件：

- *execute*: *init* 进程必须对 */bin/sh* 文件有执行权限；
- *entrypoint*: *shell\_t* 类型必须对 */bin/sh* 文件有 *entrypoint* 权限；
- *transition*: *init\_t* 类型进程必须被允许向 *shell\_t* 类型进程转换权限。

安全上下文转换语法如下：

```
#进程安全上下文转换语法
type_transition source_type temp_type : class target_type
type_transition: 类型转换关键字
source_type: 原类型
class: 类别定义关键字
target_type: 目标类型
temp_type: 对于进程转换来说就是文件类型
示例:
type_transition init_t shell_exec:process shell_t;
allow init_t shell_exec:file { getattr open read execute };#满足第一个 execute 权限条件
allow shell_t shell_exec:file { entrypoint open read execute getattr };#满足第二个 entrypoint 权限
allow init_t shell_t:process transition;#满足第三个 transition 权限
注意: 对于这种语句比较多的策略定义，移远通信已定义了宏操作，可以一步完成，例如:
domain_auto_trans(init_t,shell_exec,shell_t)
```

## 6 策略编写及使用实例

本章节以 SDK 包 `ql-ol-extsdk-ec200acntar02a03m2g_ocpu.tar.gz` 为例介绍 SELinux 编写规则；以需要完成如下策略要求的编写为例：

1. 将 `/bin/test_daemon` 设置为 `test_daemon_exec` 类型；
2. `test_daemon` 对类型为 `data_file_type` 的文件有访问权限，无读取权限；
3. `/bin/sh` 对 `test_daemon_exec` 只有访问权限，没有执行权限。

移远通信已经完成 SELinux 策略的框架，因此无需关注用户、角色和多级访问等，仅根据以下章节所述步骤完成上述策略编写即可。

### 备注

如果编译过程出现错误，说明以上定义的类型可能被使用过，可以自行定义其他类型。

### 6.1. 设置 SELinux 配置文件

SELinux 配置文件路径为 `ql-ol-extsdk-ec200acntar02a03m2g_ocpu/ql-selinux/config`。移远通信仅提供如下默认配置：`SELINUX` 默认配置为 `permissive`，`SELINUXTYPE` 默认配置为 `quectel`。可根据需要参考第3章自行修改配置值。

### 6.2. 文件标签编写

文件标签配置文件路径为 `ql-ol-extsdk-ec200acntar02a03m2g_ocpu/ql-selinux/file_contexts`。在该文件中，增加如下内容：

|                               |                 |                                             |
|-------------------------------|-----------------|---------------------------------------------|
| <code>/bin/sh</code>          | <code>-l</code> | <code>u:object_r:shell_exec:s0</code>       |
| <code>/bin/test_daemon</code> | <code>--</code> | <code>u:object_r:test_daemon_exec:s0</code> |
| <code>/run/test_file</code>   | <code>--</code> | <code>u:object_r: data_file_type:s0</code>  |

## 6.3. 策略规则文件编写

### 6.3.1. 类型文件编写

类型文件路径为 `ql-ol-extsdk-ec200acntar02a03m2g_ocpu/ql-selinux/type/file_type`。在该文件中，增加如下内容：

```
type data_file_type, exec_type, file_type;
```

### 6.3.2. TE 规则文件编写

策略文件路径为 `ql-ol-extsdk-ec200acntar02a03m2g_ocpu/ql-selinux/app/`。TE 规则文件编写步骤如下：

1. 在 `ql-ol-extsdk-ec200acntar02a03m2g_ocpu/ql-selinux/app/quectel` 路径下，创建 `test_daemon.te` 文件，内容如下：

```
#创建 test_daemon 类型，这是 test_daemon 成功运行后的进程类型
type test_daemon, domain, mlstrustedsubject;
#创建 test_daemon 文件类型为 test_daemon_exec
type test_daemon_exec, exec_type, file_type;

init_daemon_domain(test_daemon)
allow test_daemon data_file_type:file{open};
```

2. 在 `ql-ol-extsdk-ec200acntar02a03m2g_ocpu/ql-selinux/app/system/` 目录下，修改 `shell.te` 文件，增加如下内容：

```
allow shell test_daemon_exec:file{getattr};
```

## 6.4. 编译及使用

### 6.4.1. 编译文件系统镜像

在 SDK 根目录下，使用 `make rootfs` 可直接编译出含有 SELinux 策略的文件系统镜像，生成位置为 `ql-ol-extsdk-ec200acntar02a03m2g_ocpu/target` 目录下。

### 6.4.2. 烧录进模块

将 `ql-ol-extsdk-ec200acntar02a03m2g_ocpu/target` 目录下编译生成的 `root.squashfs` 文件拷贝到固件

版本包中，然后通过固件升级工具将固件烧录进模块中（可参考文档 [1]）。

## 6.5. 检查策略是否生效

系统开机后，通过如下步骤检查策略是否生效：

1. 在系统的/bin 目录下，执行 **ls -laZ** 查看 *test\_daemon* 上下文类型是否为 *test\_daemon\_exec* 类型，如下：

```
-rwxr-xr-x 1 xxx xxx u:object_r:test_daemon_exec:s0 xxx xx xx xxxx test_daemon
```

2. 通过 **dmesg|grep avc** 查看 avc log，确认 *test\_daemon* 对 *data\_file\_type* 类型的文件是否无读取权限，如下：

```
[ 89.057338] type=1400 audit(315964856.019:6): avc: denied { read } for pid=1697 comm="sh"
name="87" dev="proc" ino=7819 scontext=u:r:test_daemon:s0
tcontext=u:object_r:data_file_type:s0 tclass=file permissive=1
```

如上述标记为蓝色的内容所示，*scontext* 是主体进程的安全上下文，*tcontext* 是客体资源的安全上下文，*tclass* 是客体类别。*denied { read }* 中 *read* 是被拒绝的操作。因此，可以得出，*test\_daemon* 类型的进程（由 *scontext* 决定）对 *data\_file\_type* 类型（由 *tcontext* 决定）的文件（由 *tclass* 决定）缺少 *read*（由 *denied* 决定）权限。此时若有需求变更，例如需要 *test\_daemon* 对 *data\_file\_type* 类型的文件具有读取权限，那么需要增加策略：**allow test\_daemon data\_file\_type:file read**。

3. 执行 *./test\_daemon* 后，通过 **dmesg|grep avc** 查看 avc log，确认 */bin/sh* 对 *test\_daemon\_exec* 是否只有访问权限，没有执行权限；如下：

```
[ 391.144781] audit: type=1400 audit(1603883212.531:198): avc: denied { execute } for
pid=2602 comm="sh" name="test_daemon" dev="ubiblock0_0" ino=113 scontext=u:r:shell:s0
tcontext=u:object_r:test_daemon_exec:s0 tclass=file permissive=1
[ 391.148476] audit: type=1400 audit(1603883212.531:199): avc: denied { read } for
pid=2602 comm="sh" path="/bin/test_daemon" dev="ubiblock0_0" ino=113 scontext=u:r:shell:s0
tcontext=u:object_r:test_daemon_exec:s0 tclass=file permissive=1
[ 391.200866] audit: type=1400 audit(1603883212.561:200): avc: denied
{ execute_no_trans } for pid=2602 comm="sh" path="/bin/test_daemon" dev="ubiblock0_0"
ino=113 scontext=u:r:shell:s0 tcontext=u:object_r:test_daemon_exec:s0 tclass=file permissive=1
```

上面的 avc log 表明 shell 进程对 *test\_daemon\_exec* 有访问权限，但是没有 *execute*、*read*、*execute\_no\_trans* 权限；表明策略已成功生效。

# 7 附录 参考文档及术语缩写

表 1: 参考文档

| 文档名称                                      |
|-------------------------------------------|
| [1] Quectel_EC200A-CN(TA)_QuecOpen_快速开发指导 |

表 2: 术语缩写

| 缩写      | 英文全称                     | 中文全称        |
|---------|--------------------------|-------------|
| IoT     | Internet of Things       | 物联网         |
| SDK     | Software Development Kit | 软件开发工具包     |
| SELinux | Security-Enhanced Linux  | 安全增强型 Linux |
| TE      | Type Enforcement         | 类型强制        |